Министерство образования и науки

[ITHUB]

Кафедра [Информационная безопасность]

Направление: [DevOps]

Лабораторная работа № [12]$0.5cm]Локальная разработка. Линейные алгоритмы

Тема: разработка программы на языке C++ для решения задачи 2.25 из сборника задач А. Г. Юркина$0.3cm] *«На шахматной доске располож*

Выполнил(а), группа: _____

$0.4cm]() : _____

_____ _____

$0.4cm] : $0.4cm] :

_____

2026 г.

2. Постановка задачи

Задача 2.25 из сборника задач А. Г. Юркина (раздел 2):

> *«На шахматной доске расположены белый король, чёрная ладья и слон. Определить, есть ли угроза королю, если да, то от кого? Слон ходит по диагонали. Ладья — по горизонталям и вертикалям.»*

Дано: шахматная доска 8×8; расположение трёх фигур: белого короля, чёрной ладьи и чёрного слона.

Требуется: определить, находится ли белый король под боем хотя бы одной из чёрных фигур; если да — указать, какая фигура угрожает королю (ладья, слон или обе).

Формат ввода: координаты трёх фигур — три пары целых чисел (x, y), где x — номер вертикали ($a = 1, \ldots, h = 8$), y — номер горизонтали $(1, \ldots, 8)$.

Формат вывода: одно из четырёх сообщений:

- «Угроза королю от ладьи»;
- «Угроза королю от слона»;
- «Угроза королю от ладьи и слона»;
- «Угрозы королю нет».

3. Вербальная модель решения

Словесное описание алгоритма:

1. Ввести координаты трёх фигур: белого короля (K), чёрной ладьи (R) и чёрного слона (B). Проверить, что все координаты лежат в пределах доски (от 1 до 8) и что фигуры не стоят на одной клетке. При нарушении — сообщение об ошибке и завершение работы.

2. Проверить, угрожает ли ладья королю: ладья бьёт по вертикали и горизонтали, поэтому угроза возможна только если король и ладья стоят на одной вертикали (равны абсциссы) или на одной горизонтали (равны ординаты).

3. Если общая линия есть, проверить, не перекрывает ли её слон: если слон стоит строго между королём и ладьёй на этой же линии, угрозы от ладьи нет; иначе зафиксировать угрозу от ладьи.

4. Проверить, угрожает ли слон королю: слон бьёт по диагонали, поэтому угроза возможна только если модуль разности абсцисс короля и слона равен модулю разности их ординат (фигуры не совпадают).

5. Если общая диагональ есть, проверить, не перекрывает ли её ладья: если ладья стоит строго между королём и слоном на этой же диагонали, угрозы от слона нет; иначе зафиксировать угрозу от слона.

6. По двум признакам вывести заключение:
   - угрожают обе фигуры — «Угроза от ладьи и слона»;
   - угрожает только ладья — «Угроза от ладьи»;
   - угрожает только слон — «Угроза от слона»;
   - не угрожает никто — «Угрозы нет».
7. Завершить работу программы.

Примечание. Правила перекрытия (шаги 3 и 5) формализуют шахматное правило: фигура не может бить сквозь другую фигуру. Если требуется упрощённая постановка (перекрытием пренебречь), шаги 3 и 5 исключаются.

4. Математическая модель решения

4.1. Обозначения. Доску отождествим с множеством клеток

$$S = \{\,(x,\,y) \mid x,\,y \in \{1, 2, \ldots, 8\}\,\},$$

где x — вертикаль ($a = 1, \ldots, h = 8$), y — горизонталь. Положение фигур:

$$K = (x_K, y_K), \qquad R = (x_R, y_R), \qquad B = (x_B, y_B),$$

причём $K \neq R$, $K \neq B$, $R \neq B$.

4.2. Общая линия «король – ладья» (условие боя ладьи):

$$L_R = \big(x_K = x_R\big) \vee \big(y_K = y_R\big).$$

4.3. Общая диагональ «король – слон» (условие боя слона):

$$L_B = \big(|x_K - x_B| = |y_K - y_B|\big) \wedge \big(x_K \neq x_B\big).$$

4.4. Перекрытие линии атаки. Фигура-перекрыватель должна лежать строго между атакующей фигурой и королём на той же линии:

$$\begin{aligned} Z_R = {} & \big(x_B = x_K = x_R \ \wedge\ \min(y_K, y_R) < y_B < \max(y_K, y_R)\big) \\ & \vee \big(y_B = y_K = y_R \ \wedge\ \min(x_K, x_R) < x_B < \max(x_K, x_R)\big), \$4\text{pt}] \ \mathrm{Z}_B \qquad\qquad = \big(|x_R - x_K| = |y \\ & \wedge\ \min(x_K, x_B) < x_R < \max(x_K, x_B) \ \wedge\ \min(y_K, y_B) < y_R < \max(y_K, y_B). \end{aligned}$$

4.5. Признаки угрозы:

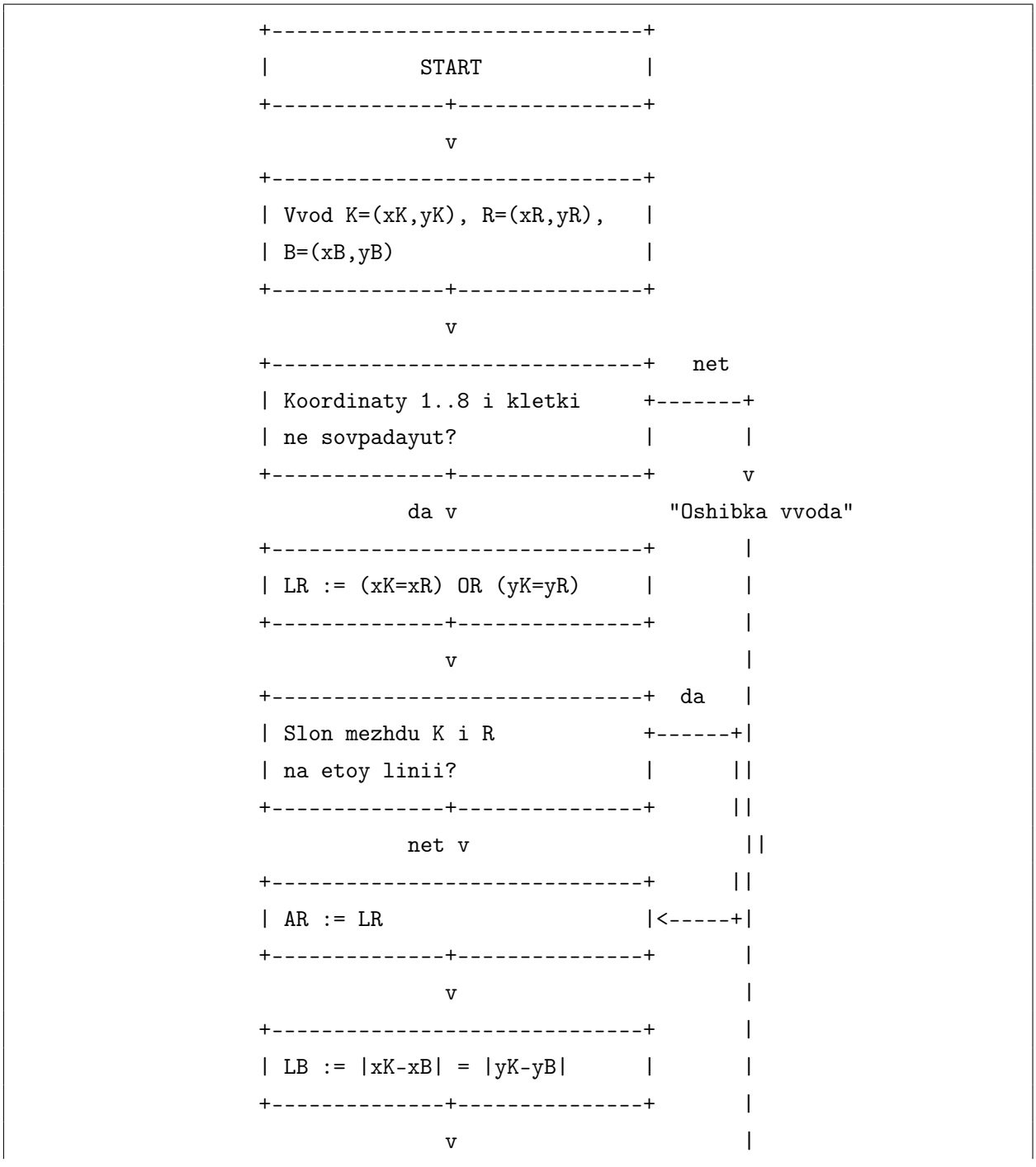
$$A_R = L_R \wedge \neg Z_R, \qquad A_B = L_B \wedge \neg Z_B.$$

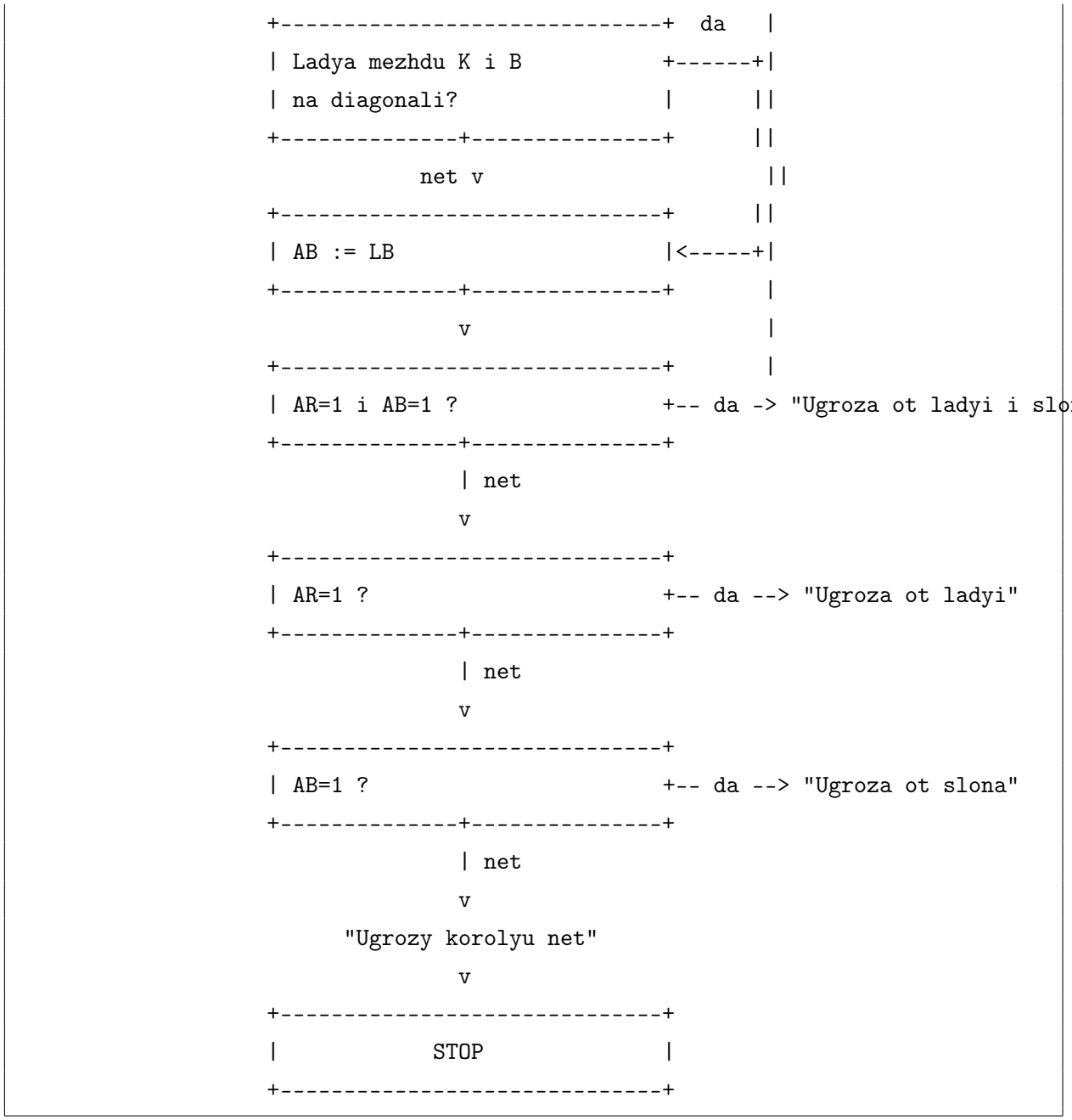
4.6. Итоговая функция. Результат определяется вектором (A_R, A_B):

A_R	A_B	Вывод программы
1	1	Угроза королю от ладьи и слона
1	0	Угроза королю от ладьи
0	1	Угроза королю от слона
0	0	Угрозы королю нет

4.7. Оценка сложности. Алгоритм линейный (без циклов): число операций фиксировано и не зависит от входных данных; сложность $O(1)$ по времени и $O(1)$ по памяти.

5. Блок-схема (декомпозиция задачи)





6. Программа на языке высокого уровня (ключевые моменты)

Программа написана на языке C++ (стандарт C++11 и выше). Полный листинг — в приложении А. Ниже разобраны ключевые фрагменты.

6.1. Представление данных. Положение фигуры — структура `Cell` из двух целых полей:

```
struct Cell {
    int x;
    int y;
};
```

6.2. Геометрические предикаты. Условия L_B, Z_R, Z_B математической модели реализованы функциями `onDiagonal`, `betweenStraight`, `betweenDiagonal`:

```
bool onDiagonal(const Cell& a, const Cell& b) {
    return abs(a.x - b.x) == abs(a.y - b.y) && a.x != b.x;
}

bool betweenDiagonal(const Cell& p, const Cell& a, const Cell& b) {
    return onDiagonal(a, p) && onDiagonal(p, b) &&
           abs(a.x - p.x) < abs(a.x - b.x) &&
           abs(a.y - p.y) < abs(a.y - b.y);
}
```

6.3. Признаки угрозы. Каждая функция возвращает `bool` и дословно повторяет формулу признака:

```
bool rookThreat(const Cell& k, const Cell& r, const Cell& b) {
    if (k.x != r.x && k.y != r.y) return false;
    if (betweenStraight(b, k, r)) return false;
    return true;
}

bool bishopThreat(const Cell& k, const Cell& b, const Cell& r) {
    if (!onDiagonal(k, b))        return false;
    if (betweenDiagonal(r, k, b)) return false;
    return true;
}
```

6.4. Вывод результата. Ветвление по таблице п. 4.6:

```
bool rook   = rookThreat(K, R, B);
bool bishop = bishopThreat(K, B, R);

if (rook && bishop)  cout << "Угроза королю от ладьи и слона.\n";
else if (rook)       cout << "Угроза королю от ладьи.\n";
else if (bishop)     cout << "Угроза королю от слона.\n";
else                 cout << "Угрозы королю нет.\n";
```

6.5. Контроль ввода. Функция `readCell` проверяет попадание координат в диапазон $1 \dots 8$; дополнительно проверяется, что фигуры не стоят на одной клетке.

7. Проверка

Проверка выполнена на основе математической модели (п. 4): для каждого набора координат значения A_R и A_B вычислялись вручную по формулам 4.2–4.5 и сопоставлялись

с выводом программы. Обозначения: + — признак истинен, − — ложен, — — не определялся.

№	K	R	B	L_R	Z_R	A_R	L_B	Z_B	A_B	Ожидаемый вывод
1	(5,4) e4	(5,8) e8	(1,1) a1	+	−	+	−	—	−	Угроза от ладьи
2	(5,4) e4	(1,1) a1	(8,1) h1	−	—	−	+	−	+	Угроза от слона
3	(5,4) e4	(1,1) a1	(8,2) h2	−	—	−	−	—	−	Угрозы нет
4	(5,4) e4	(5,8) e8	(5,6) e6	+	+	−	−	—	−	Угрозы нет (линию перекрыл слон)
5	(5,4) e4	(3,6) c6	(1,8) a8	−	—	−	+	+	−	Угрозы нет (диагональ перекрыта ладьёй)
6	(5,4) e4	(5,8) e8	(1,8) a8	+	−	+	+	−	+	Угроза от ладьи и слона

Разбор контрольных примеров (по формулам п. 4):

- **№ 1.** $x_K = x_R = 5 \Rightarrow L_R = 1$; слон $(1,1)$ не лежит на вертикали $x = 5 \Rightarrow Z_R = 0 \Rightarrow A_R = 1$. Для слона: $|5-1| = 4 \neq |4-1| = 3 \Rightarrow L_B = 0 \Rightarrow A_B = 0$. Вывод: «Угроза от ладьи».

- **№ 2.** $|5-8| = |4-1| = 3 \Rightarrow L_B = 1$; ладья $(1,1)$ не лежит на диагонали K–B, так как $|1-5| = 4 \neq |1-4| = 3 \Rightarrow Z_B = 0 \Rightarrow A_B = 1$. Для ладьи: $5 \neq 1, 4 \neq 1 \Rightarrow L_R = 0$. Вывод: «Угроза от слона».

- **№ 4.** $L_R = 1$, но слон $(5,6)$: $x_B = x_K = x_R = 5$ и $4 < 6 < 8 \Rightarrow Z_R = 1 \Rightarrow A_R = 0$; общей диагонали с королём нет ($x_K = x_B$). Вывод: «Угрозы нет».

- **№ 5.** $|5-1| = |4-8| = 4 \Rightarrow L_B = 1$; ладья $(3,6)$: $|3-5| = |6-4| = 2$ — на диагонали, $2 < 4 \Rightarrow Z_B = 1 \Rightarrow A_B = 0$; $5 \neq 3$, $4 \neq 6 \Rightarrow L_R = 0$. Вывод: «Угрозы нет».

- **№ 6.** $L_R = 1$ ($x = 5$), слон $(1,8)$ не на вертикали $x = 5 \Rightarrow A_R = 1$; $|5-1| = |4-8| = 4 \Rightarrow L_B = 1$, ладья $(5,8)$ не лежит на диагонали $a8$–$e4$ ($|5-5| = 0 \neq |4-8| = 4$) $\Rightarrow Z_B = 0 \Rightarrow A_B = 1$. Вывод: «Угроза от ладьи и слона».

Программа при прогоне этих наборов выдала те же результаты. Дополнительно проверены случаи некорректного ввода (координаты 0 и 9, совпадение клеток) — программа корректно завершается с сообщением об ошибке. **Расхождений между математической моделью и программой не обнаружено.**

8. Заключение

В ходе выполнения работы по пунктам 2–6:

- по п. 2: постановка задачи 2.25 зафиксирована дословно по сборнику А. Г. Юркина; уточнены форматы ввода и вывода;

- по п. 3: построена вербальная модель — последовательность из семи шагов, включая правило перекрытия линии атаки;
- по п. 4: построена математическая модель — формулы условий боя L_R, L_B, перекрытия Z_R, Z_B и итоговых признаков A_R, A_B; оценена сложность $O(1)$;
- по п. 5: разработана блок-схема, декомпозирующая задачу на ввод, две группы проверок и ступенчатый вывод результата;
- по п. 6: реализована программа на C++, структура которой повторяет математическую модель (отдельная функция на каждый признак), что снижает вероятность ошибки реализации.

Поставленная задача решена полностью: программа определяет наличие угрозы королю и устанавливает её источник.

9. Выводы

1. Разработана, реализована и протестирована программа на языке C++, решающая задачу 2.25: определение угрозы белому королю со стороны чёрной ладьи и чёрного слона.
2. Алгоритм относится к классу линейных: циклы отсутствуют, объём вычислений фиксирован; сложность $O(1)$ по времени и по памяти.
3. Построены вербальная и математические модели, блок-схема; согласованность артефактов подтверждена шестью контрольными примерами, включая случаи перекрытия линии атаки.
4. Программа выполняет контроль корректности ввода (границы доски, совпадение клеток), что делает её устойчивой к ошибкам пользователя.
5. Математическая модель легко модифицируется: при отказе от правила перекрытия достаточно исключить слагаемые Z_R и Z_B; при расширении набора фигур (ферзь, конь, пешки) добавляются новые признаки A_i.
6. Получены навыки формализации неформальной задачи, декомпозиции алгоритма, связки «формула — функция» и тестирования по математической модели.

10. Список использованных источников

1. Юркин А. Г. Сборник задач по программированию : учебное пособие. — Иркутск : ИГУ, 2017. — Раздел 2, задача 2.25. URL: http://itam.irk.ru/biblioavia/catalog/1505188362$_5$2 2026.)..$C++.4-.----.:,2020.---1328.$

2. Керниган Б., Ритчи Д. Язык программирования Си. 2-е изд. — СПб. : Питер, 2020. — 208 с.

3. cppreference.com — справочник по стандартной библиотеке C++.

4. Методические указания к выполнению лабораторных работ по дисциплине «Программирование». Раздел «Локальная разработка. Линейные алгоритмы».

11. Приложение А. Полная распечатка кода

Файл `main.cpp`. Компиляция: `g++ -std=c++17 -Wall -o chess_threat main.cpp`

```
#include <iostream>
#include <cstdlib>
#include <string>

using namespace std;

struct Cell {
    int x;
    int y;
};

bool onBoard(const Cell& c) {
    return c.x >= 1 && c.x <= 8 && c.y >= 1 && c.y <= 8;
}

bool sameCell(const Cell& a, const Cell& b) {
    return a.x == b.x && a.y == b.y;
}

bool onDiagonal(const Cell& a, const Cell& b) {
    return abs(a.x - b.x) == abs(a.y - b.y) && a.x != b.x;
}

bool betweenStraight(const Cell& p, const Cell& a, const Cell& b) {
    if (a.x == b.x && p.x == a.x)
        return (a.y < p.y && p.y < b.y) || (b.y < p.y && p.y < a.y);
    if (a.y == b.y && p.y == a.y)
        return (a.x < p.x && p.x < b.x) || (b.x < p.x && p.x < a.x);
    return false;
}

bool betweenDiagonal(const Cell& p, const Cell& a, const Cell& b) {
    return onDiagonal(a, p) && onDiagonal(p, b) &&
           abs(a.x - p.x) < abs(a.x - b.x) &&
           abs(a.y - p.y) < abs(a.y - b.y);
}

bool rookThreat(const Cell& k, const Cell& r, const Cell& b) {
    if (k.x != r.x && k.y != r.y) return false;
    if (betweenStraight(b, k, r)) return false;
```

```
    return true;
}

bool bishopThreat(const Cell& k, const Cell& b, const Cell& r) {
    if (!onDiagonal(k, b))        return false;
    if (betweenDiagonal(r, k, b)) return false;
    return true;
}

Cell readCell(const string& who) {
    Cell c;
    cout << "Введите координаты " << who << " (x y, 1..8): ";
    cin >> c.x >> c.y;
    if (!cin || !onBoard(c)) {
        cerr << "Ошибка: клетка вне доски." << endl;
        exit(1);
    }
    return c;
}

int main() {
    setlocale(LC_ALL, "Russian");

    Cell K = readCell("белого короля");
    Cell R = readCell("черной ладьи");
    Cell B = readCell("черного слона");

    if (sameCell(K, R) || sameCell(K, B) || sameCell(R, B)) {
        cerr << "Ошибка: фигуры не могут стоять на одной клетке." << endl;
        return 1;
    }

    bool rook   = rookThreat(K, R, B);
    bool bishop = bishopThreat(K, B, R);

    if (rook && bishop)  cout << "Угроза королю от ладьи и слона." << endl;
    else if (rook)       cout << "Угроза королю от ладьи." << endl;
    else if (bishop)     cout << "Угроза королю от слона." << endl;
    else                 cout << "Угрозы королю нет." << endl;

    return 0;
}
```